第 3 章 马尔可夫决策过程

3.1 简介

马尔可夫决策过程（Markov decision process，MDP）是强化学习的重要概念。要学好强化学习，我们首先要掌握马尔可夫决策过程的基础知识。前两章所说的强化学习中的环境一般就是一个马尔可夫决策过程。与多臂老虎机问题不同，马尔可夫决策过程包含状态信息以及状态之间的转移机制。如果要用强化学习去解决一个实际问题，第一步要做的事情就是把这个实际问题抽象为一个马尔可夫决策过程，也就是明确马尔可夫决策过程的各个组成要素。本章将从马尔可夫过程出发，一步一步地进行介绍，最后引出马尔可夫决策过程。

3.2 马尔可夫过程

3.2.1 随机过程

随机过程（stochastic process）是概率论的“动力学”部分。概率论的研究对象是静态的随机现象，而随机过程的研究对象是随时间演变的随机现象（例如天气随时间的变化、城市交通随时间的变化）。在随机过程中，随机现象在某时刻的取值是一个向量随机变量，用表示，所有可能的状态组成状态集合S。随机现象便是状态的变化过程。在某时刻的状态通常取决于时刻之前的状态。我们将已知历史信息$(S_1,\ldots,S_t)$时下一个时刻状态为S_{t+1}的概率表示成$P(S_{t+1}|S_1,\ldots,S_t)$。

3.2.2 马尔可夫性质

当且仅当某时刻的状态**只取决于上一时刻的状态**时，一个随机过程被称为具有**马尔可夫性质**（Markov property），用公式表示为$P(S_{t+1}|S_t)=P(S_{t+1}|S_1,\ldots,S_t)$。也就是说，当前状态是未来的充分统计量，即下一个状态只取决于当前状态，而不会受到过去状态的影响。需要明确的是，具有马尔可夫性并不代表这个随机过程就和历史完全没有关系。因为虽然$t+1$时刻的状态只与t时刻的状态有关，但是t时刻的状态其实包含了$t-1$时刻的状态的信息，通过这种链式的关系，历史的信息被传递到了现在。马尔可夫性可以大大简化运算，因为只要当前状态可知，所有的历史信息都不再需要了，利用当前状态信息就可以决定未来。

3.2.3 马尔可夫过程

马尔可夫过程（Markov process）指具有马尔可夫性质的随机过程，也被称为**马尔可夫链**（Markov chain）。我们通常用元组$<S,P>$描述一个马尔可夫过程，其中S是有限数量的状态集合，P是**状态转移矩阵**（state transition matrix）。假设一共有n个状态，此时$S=\{s_1,s_2,\ldots\ldots,s_n\}$。状态转移矩阵$P$定义了所有状态对之间的转移概率，即

$$\mathcal{P}=\begin{bmatrix} P(s_1|s_1) & \cdots & P(s_n|s_1) \\ \vdots & \ddots & \vdots \\ P(s_1|s_n) & \cdots & P(s_n|s_n) \end{bmatrix}$$

矩阵P中第i行第j列元素$P(s_j|s_i)=P(S_{t+1}=s_j|S_t=s_j)$表示从状态转移到状态的概率，我们称为$P(s'|s)$状态转移函数。从某个状态出发，到达其他状态的概率和必须为 1，即状态转移矩阵的每一行的和为 1。

图 3-1 是一个具有 6 个状态的马尔可夫过程的简单例子。其中每个绿色圆圈表示一个状态，每个状态都有一定概率（包括概率为 0）转移到其他状态，其中s_6通常被称为**终止状态**（terminal state），因为它不会再转移到其他状态，可以理解为它永远以概率 1 转移到自己。状态之间的虚线箭头表示状态的转移，箭头旁的数字表示该状态转移发生的概率。从每个状态出发转移到其他状态的概率总和为 1。例如，s_1有 90%概率保持不变，有 10%概率转移到s_2，而在s_2又有 50%概率回到s_1，有 50%概率转移到s_3。

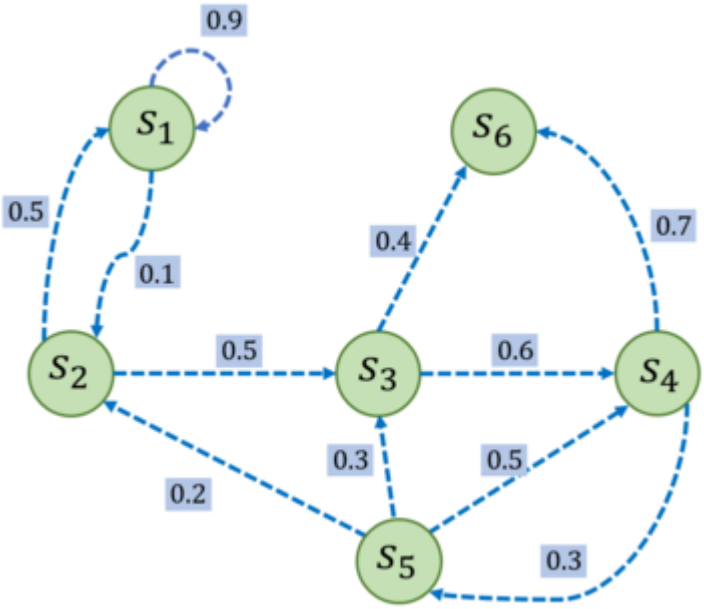

图3-1 马尔可夫过程的一个简单例子

我们可以写出这个马尔可夫过程的状态转移矩阵：

$$\mathcal{P}=\begin{bmatrix} 0.9 & 0.1 & 0 & 0 & 0 & 0 \\ 0.5 & 0 & 0.5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.6 & 0 & 0.4 \\ 0 & 0 & 0 & 0 & 0.3 & 0.7 \\ 0 & 0.2 & 0.3 & 0.5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

其中第i行j列的$P_{i,j}$值则代表从状态s_i转移到s_j的概率。

给定一个马尔可夫过程，我们就可以从某个状态出发，根据它的状态转移矩阵生成一个状态**序列**（episode），这个步骤也被叫做**采样**（sampling）。例如，从s_1出发，可以生成序列$s_1 \to s_2 \to s_3 \to s_6$或序列$s_1 \to s_1 \to s_2 \to s_3 \to s_4 \to s_5 \to s_3 \to s_6$等。生成这些序列的概率和状态转移矩阵有关。

3.3 马尔可夫奖励过程

在马尔可夫过程的基础上加入奖励函数 和折扣因子，就可以得到**马尔可夫奖励过程**（Markov reward process）。一个马尔可夫奖励过程由$< S, P, r, \gamma >$构成，各个组成元素的含义如下所示。

- S是有限状态的集合。
- P是状态转移矩阵。
- r是奖励函数，某个状态s的奖励$r(s)$指转移到该状态时可以获得奖励的期望。
- γ是折扣因子（discount factor），γ的取值范围为$[0, 1)$。引入折扣因子的理由为远期利益具有一定不确定性，有时我们更希望能够尽快获得一些奖励，所以我们需要对远期利益打一些折扣。接近 1 的γ更关注长期的累计奖励，接近 0 的γ更考虑短期奖励。

3.3.1 回报

在一个马尔可夫奖励过程中，从第t时刻状态S_t开始，直到终止状态时，所有奖励的衰减之和称为**回报**G_t（Return），公式如下：

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

其中，R_t表示在时刻获得的奖励。在图 3-2 中，我们继续沿用图 3-1 马尔可夫过程的例子，并在其基础上添加奖励函数，构建成一个马尔可夫奖励过程。例如，进入状态s_2可以得到奖励-2，表明我们不希望进入s_2，进入s_4可以获得最高的奖励10，但是进入s_6之后奖励为零，并且此时序列也终止了。

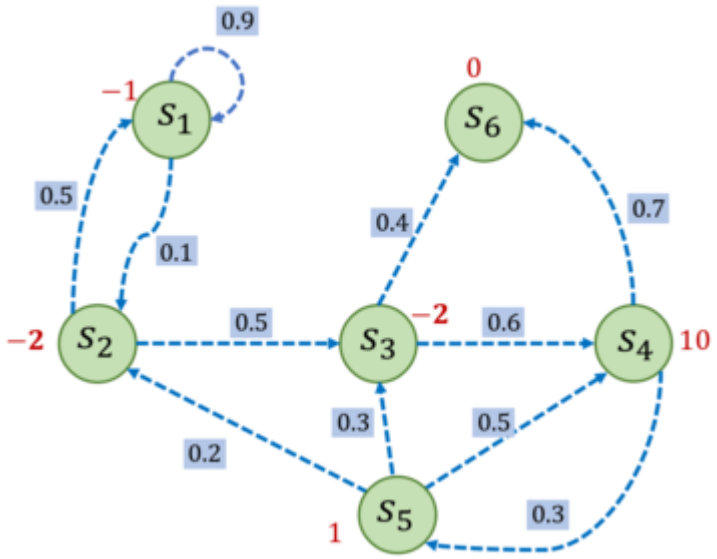

图3-2 马尔可夫奖励过程的一个简单例子

比如选取s_1为起始状态，设置$\gamma = 0.5$，采样到一条状态序列为$s_1 \to s_2 \to s_3 \to s_6$，就可以计算$s_1$的回报$G_1$，得到$G_1 = -1 + 0.5 \times (-2) + 0.5^2 \times (-2) = -2.5$。

接下来我们用代码表示图 3-2 中的马尔可夫奖励过程，并且定义计算回报的函数。

```python
import numpy as np
np.random.seed(0)
# 定义状态转移概率矩阵P
P = [
    [0.9, 0.1, 0.0, 0.0, 0.0, 0.0],
    [0.5, 0.0, 0.5, 0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.6, 0.0, 0.4],
    [0.0, 0.0, 0.0, 0.0, 0.3, 0.7],
    [0.0, 0.2, 0.3, 0.5, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.0, 0.0, 1.0],
]
P = np.array(P)

rewards = [-1, -2, -2, 10, 1, 0]  # 定义奖励函数
gamma = 0.5  # 定义折扣因子


# 给定一条序列,计算从某个索引（起始状态）开始到序列最后（终止状态）得到的回报
def compute_return(start_index, chain, gamma):
    G = 0
    //// 从序列的末尾开始向前遍历
    for i in reversed(range(start_index, len(chain))):
        //// 每一步的回报 = 折扣后的未来回报 + 当前状态的即时奖励
        G = gamma * G + rewards[chain[i] - 1]
    return G


# 一个状态序列,s1-s2-s3-s6
chain = [1, 2, 3, 6]
start_index = 0
G = compute_return(start_index, chain, gamma)
print("根据本序列计算得到回报为：%s。" % G)
```

`根据本序列计算得到回报为：-2.5`

3.3.2 价值函数 （未完待续！！！）

在马尔可夫奖励过程中，一个状态的期望回报（即从这个状态出发的未来累积奖励的期望）被称为这个状态的**价值**（value）。所有状态的价值就组成了**价值函数**（value function），价值函数的输入为某个状态，输出为这个状态的价值。我们将价值函数写成$V(s)=E[G_t|S_t=s]$，展开为

$$
\begin{aligned}
V(s) &= \mathbb{E}[G_t|S_t=s] \\
&= \mathbb{E}[R_t+\gamma R_{t+1}+\gamma^2 R_{t+2}+\ldots|S_t=s] \\
&= \mathbb{E}[R_t+\gamma(R_{t+1}+\gamma R_{t+2}+\ldots)|S_t=s] \\
&= \mathbb{E}[R_t+\gamma G_{t+1}|S_t=s] \\
&= \mathbb{E}[R_t+\gamma V(S_{t+1})|S_t=s]
\end{aligned}
$$

在上式的最后一个等号中，一方面，即时奖励的期望正是奖励函数的输出，即$E[R_t|S_t=s]=r(s)$；另一方面，等式中剩余部分$E[\gamma V(S_{t+1}|S_t=s)]$可以根据从状态出发的转移概率得到，即可以得到

$$V(s) = r(s) + \gamma \sum_{s' \in S} p(s'|s) V(s')$$

上式就是马尔可夫奖励过程中非常有名的**???贝尔曼方程???**（Bellman equation），对每一个状态都成立。若一个马尔可夫奖励过程一共有n个状态，即$S = \{s_1, s_2, \ldots, s_n\}$，我们将所有状态的价值表示成一个列向量$V = [V(s_1), V(s_2), \ldots, V(s_n)]^T$，同理，将奖励函数写成一个列向量$R = [r(s_1), r(s_2), \ldots, r(s_n)]^T$。于是我们可以将贝尔曼方程写成矩阵的形式：

$$\mathcal{V} = \mathcal{R} + \gamma \mathcal{P} \mathcal{V}$$

$$\begin{bmatrix} V(s_1) \\ V(s_2) \\ \ldots \\ V(s_n) \end{bmatrix} = \begin{bmatrix} r(s_1) \\ r(s_2) \\ \ldots \\ r(s_n) \end{bmatrix} + \gamma \begin{bmatrix} P(s_1|s_1) & p(s_2|s_1) & \ldots & P(s_n|s_1) \\ P(s_1|s_2) & P(s_2|s_2) & \ldots & P(s_n|s_2) \\ \ldots & & & \\ P(s_1|s_n) & P(s_2|s_n) & \ldots & P(s_n|s_n) \end{bmatrix} \begin{bmatrix} V(s_1) \\ V(s_2) \\ \ldots \\ V(s_n) \end{bmatrix}$$

我们可以直接根据矩阵运算求解，得到以下解析解：

$$\begin{aligned} \mathcal{V} &= \mathcal{R} + \gamma \mathcal{P} \mathcal{V} \\ (I - \gamma \mathcal{P}) \mathcal{V} &= \mathcal{R} \\ \mathcal{V} &= (I - \gamma \mathcal{P})^{-1} \mathcal{R} \end{aligned}$$

以上解析解的计算复杂度是$O(n^3)$，其中n是状态个数，因此这种方法只适用很小的马尔可夫奖励过程。求解较大规模的马尔可夫奖励过程中的价值函数时，可以使用**动态规划**（dynamic programming）算法、**蒙特卡洛方法**（Monte-Carlo method）和**时序差分**（temporal difference），这些方法将在之后的章节介绍。

接下来编写代码来实现求解价值函数的解析解方法，并据此计算该马尔可夫奖励过程中所有状态的价值。

```python
def compute(P, rewards, gamma, states_num):
    ''' 利用贝尔曼方程的矩阵形式计算解析解,states_num是MRP的状态数 '''
    rewards = np.array(rewards).reshape((-1, 1))  #将rewards写成列向量形式
    value = np.dot(np.linalg.inv(np.eye(states_num, states_num) - gamma * P),
                   rewards)
    return value


V = compute(P, rewards, gamma, 6)
print("MRP中每个状态价值分别为\n", V)
```

```
MRP中每个状态价值分别为
 [[-2.01950168]
 [-2.21451846]
 [ 1.16142785]
 [10.53809283]
 [ 3.58728554]
 [ 0.        ]]
```

根据以上代码，求解得到各个状态的价值$V(s)$，具体如下：

$$\begin{bmatrix} V(s_1) \\ V(s_2) \\ V(s_3) \\ V(s_4) \\ V(s_5) \\ V(s_6) \end{bmatrix} = \begin{bmatrix} -2.02 \\ -2.21 \\ 1.16 \\ 10.54 \\ 3.59 \\ 0 \end{bmatrix}$$

我们现在用贝尔曼方程来进行简单的验证。例如，对于状态来说，当时，有

$$V(s_4) = r(s_4) + \gamma \sum_{s' \in S} P(s'|s_4) V(s')$$

$$10.54 = 10 + 0.5 \times (0.7 \times 0 + 0.3 \times 3.59)$$

可以发现左右两边的值几乎是相等的，说明我们求解得到的价值函数是满足状态s_4为时的贝尔曼方程。读者可以自行验证在其他状态时贝尔曼方程是否也成立。若贝尔曼方程对于所有状态都成立，就可以说明我们求解得到的价值函数是正确的。除了使用动态规划算法，马尔可夫奖励过程中的价值函数也可以通过蒙特卡洛方法估计得到，我们将在 3.5 节中介绍该方法。